

LAPORAN UJIAN AKHIR SEMESTER
MATA KULIAH PEMROGRAMAN SISI SERVER (PSS)

TOKOJOKO

Backend REST API E-Commerce (Toko Bunga)
Dibangun dengan Laravel 10, Sanctum, dan MySQL

Nama Mahasiswa	:	
NIM	:	
Kelas	:	
Program Studi	:	
Repository	:	https://github.com/imdevedugame/e-commerce_new
Tanggal	:	14 Juli 2026

DAFTAR ISI

1. BAB I Latar Belakang
2. BAB II Analisa dan Rancangan Aplikasi, Pemodelan UML
 1. 2.1 Analisa Kebutuhan
 2. 2.2 Analisa Kebutuhan Hardware dan Software
 3. 2.3 Pemodelan UML
 4. 2.4 Skema Diagram Relasi Tabel
 5. 2.5 Model
 6. 2.6 Query
 7. 2.7 Import CSV
 8. 2.8 API
 9. 2.9 Auth
 10. 2.10 Throttling, Pagination, Filtering, Sorting
 11. 2.11 Docs
 12. 2.12 Unit Testing
3. BAB III Implementasi Sistem
4. BAB IV Penutup
5. Lampiran Code Program

BAB I

LATAR BELAKANG

Bisnis ritel bunga (florist) semakin banyak berpindah dari transaksi tatap muka ke penjualan daring. Aplikasi TokoJoko yang telah ada sebelumnya melayani hal ini lewat storefront berbasis Livewire yang bersifat server-rendered dan mengandalkan sesi/cookie, sehingga hanya dapat diakses lewat peramban (browser) di alamat web tertentu. Kebutuhan pasar saat ini menuntut lebih dari itu: pemilik toko ingin produknya juga dapat diakses lewat aplikasi mobile (Android/iOS) atau diintegrasikan ke aplikasi pihak ketiga lain, sesuatu yang tidak dapat dilakukan oleh arsitektur berbasis sesi/cookie karena klien eksternal tidak dapat berbagi cookie peramban dengan server.

Atas dasar itu, backend TokoJoko dikembangkan lebih lanjut dengan menambahkan lapisan REST API (`/api/v1/*`) yang bersifat *stateless* dan berbasis token, sehingga dapat dikonsumsi oleh klien apa pun — baik untuk keperluan tugas mata kuliah Pemrograman Sisi Server (PSS) ini, maupun untuk disambungkan ke aplikasi klien lain seperti tugas Pemrograman Perangkat Bergerak (PPB) atau Pemrograman Sisi Client (PSC).

Laporan ini disusun untuk mendokumentasikan proses analisa, rancangan, implementasi, dan pengujian backend tersebut secara menyeluruh, meliputi desain basis data, pemodelan UML, implementasi API lengkap dengan autentikasi, pembatasan jumlah permintaan (throttling), pagination, filtering dan sorting, fitur impor data massal melalui berkas CSV, pengujian otomatis (unit/feature test), dokumentasi API, hingga kesiapan proses hosting.

ANALISA DAN RANCANGAN APLIKASI, PEMODELAN UML

Bab ini menguraikan hasil analisa kebutuhan, rancangan basis data, dan pemodelan UML (use case diagram, class diagram, dan sequence diagram) yang menjadi dasar implementasi, dilanjutkan dengan pembahasan teknis setiap komponen backend: Model, Query, Import CSV, API, Auth, Throttling, Pagination, Filtering, Sorting, Docs, dan Unit Testing.

2.1 Analisa Kebutuhan

Kebutuhan sistem dibagi menjadi kebutuhan fungsional (fitur yang harus tersedia) dan kebutuhan non-fungsional (kualitas sistem secara keseluruhan).

Kebutuhan Fungsional

Kode	Kebutuhan
F1	Pengguna dapat mendaftar (register) dan masuk (login) untuk mendapatkan token akses API.
F2	Pengguna, baik tamu maupun yang sudah masuk, dapat melihat daftar dan detail produk beserta kategorinya, dengan kemampuan pencarian, filter harga, dan pengurutan.
F3	Pengguna yang sudah masuk dapat membuat pesanan (checkout) dengan validasi ketersediaan stok secara otomatis.
F4	Pengguna hanya dapat melihat pesanan miliknya sendiri; administrator dapat melihat seluruh pesanan.
F5	Administrator dapat mengimpor banyak data produk sekaligus melalui berkas CSV.
F6	Pengguna dapat keluar (logout), yang akan mencabut token yang sedang aktif.
F7	Sistem menyediakan dokumentasi API yang dapat dicoba langsung (Swagger UI) dan diimpor ke Postman.

Kebutuhan Non-Fungsional

Kode	Kebutuhan
N1	Keamanan — autentikasi berbasis token (Sanctum), otorisasi kepemilikan data (policy), dan validasi input pada setiap endpoint.
N2	Ketersediaan dan anti-penyalahgunaan — seluruh endpoint dibatasi jumlah permintaannya (throttling) untuk mencegah penyalahgunaan maupun serangan brute-force.
N3	Skalabilitas — API bersifat stateless (menggunakan token, bukan sesi) sehingga dapat di-deploy dan diskalakan secara terpisah dari storefront.
N4	Kualitas dan keandalan kode — dicakup oleh pengujian otomatis (unit/feature test) sehingga regresi dapat terdeteksi sejak dini.
N5	Kemudahan integrasi — didokumentasikan secara lengkap (OpenAPI/Swagger, Postman, Markdown) agar mudah dikonsumsi oleh klien lain seperti aplikasi mobile atau SPA.

2.2 Analisa Kebutuhan Hardware dan Software

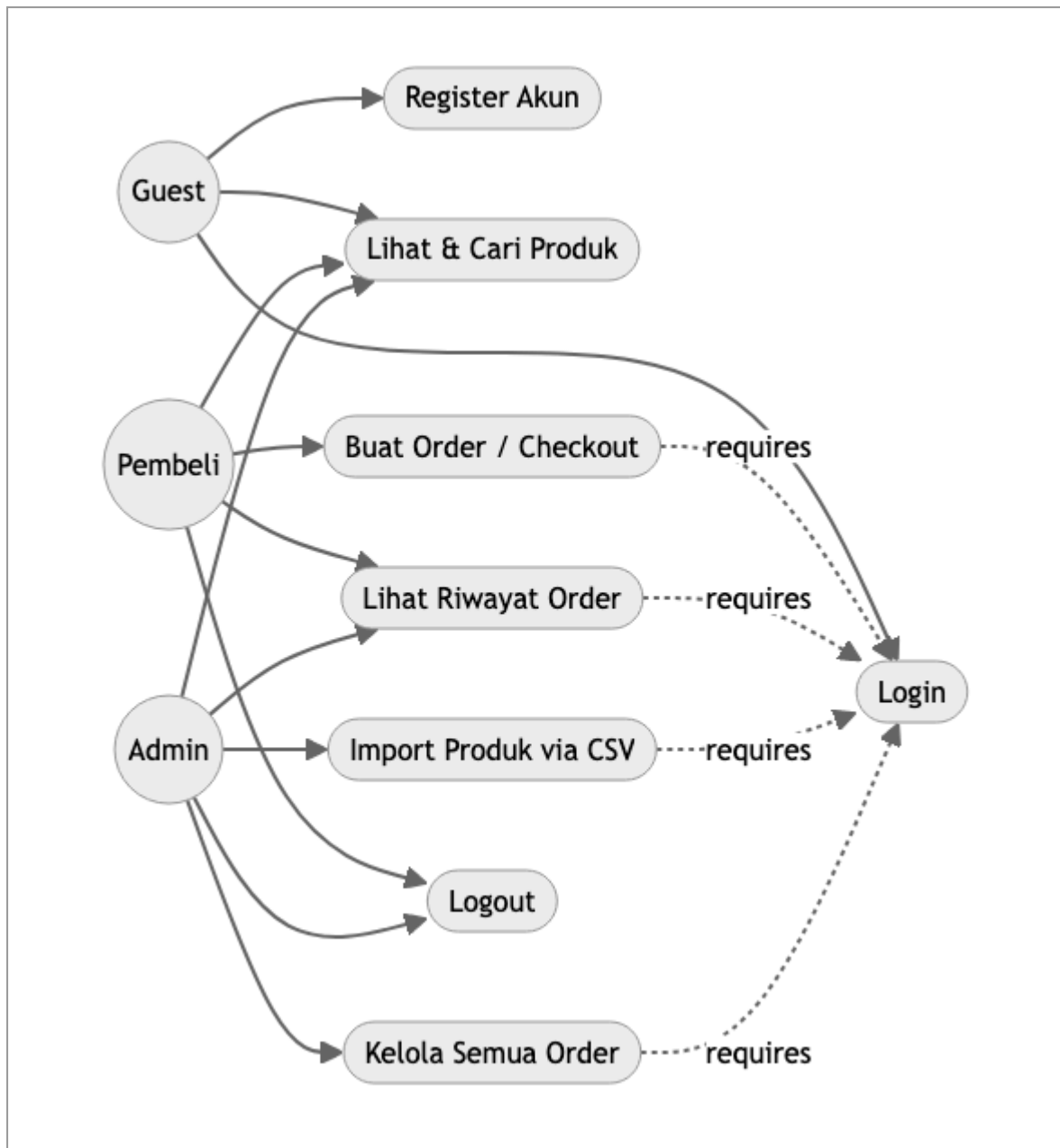
Tabel berikut merangkum kebutuhan perangkat keras dan perangkat lunak minimum, baik untuk lingkungan pengembangan maupun lingkungan produksi (hosting).

Kategori	Kebutuhan Minimum
Server (hosting)	1 vCPU, RAM 1 GB, storage 5 GB — cukup untuk menjalankan PHP-FPM dan MySQL pada skala kecil-menengah, misalnya pada VPS atau shared hosting seperti Railway, Render, atau Niagahoster.
Sistem operasi server	Linux (Ubuntu 22.04 atau setara) yang mendukung PHP 8.1 ke atas.
Perangkat lunak server	PHP 8.1+ (diuji pada PHP 8.4), Composer 2, MySQL 8 atau MariaDB 10.6+, web server Nginx/Apache, serta ekstensi PHP: mbstring, pdo_mysql, openssl, tokenizer, xml, ctype, json.
Lingkungan pengembangan	PHP 8.4, Composer, Node.js beserta npm (untuk build aset Livewire/Filament), MySQL 8 lokal, dan Git.
Perangkat klien (konsumen API)	Perangkat apa pun yang memiliki koneksi internet dan HTTP client — peramban (untuk Swagger UI), Postman, atau aplikasi mobile (Android/iOS) yang mampu memanggil REST API dan menyimpan bearer token.

2.3 Pemodelan UML

Pemodelan UML digunakan untuk menggambarkan interaksi aktor dengan sistem (use case diagram), struktur data beserta relasinya (class diagram), dan alur komunikasi antar komponen saat memproses satu skenario penuh (sequence diagram).

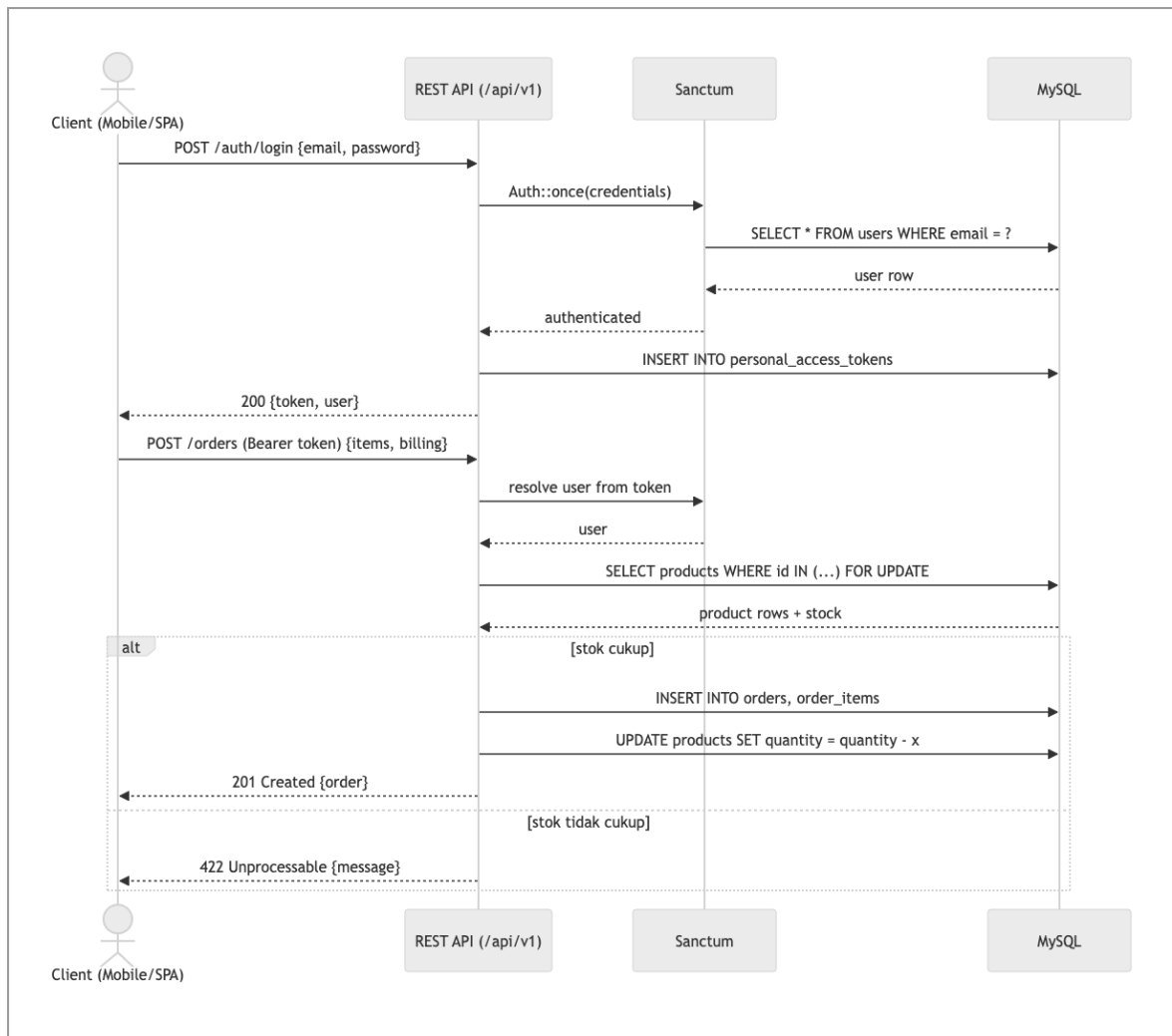
Use Case Diagram. Terdapat tiga aktor: *Guest* (pengunjung yang belum masuk), *Pembeli* (pengguna yang sudah melakukan autentikasi), dan *Admin*. Sebagian besar use case pengguna terautentikasi mensyaratkan use case *Login* terlebih dahulu, digambarkan dengan relasi *requires*.



Class Diagram. Diagram berikut menggambarkan enam model Eloquent utama beserta atribut, method, relasi, dan multiplisitasnya.

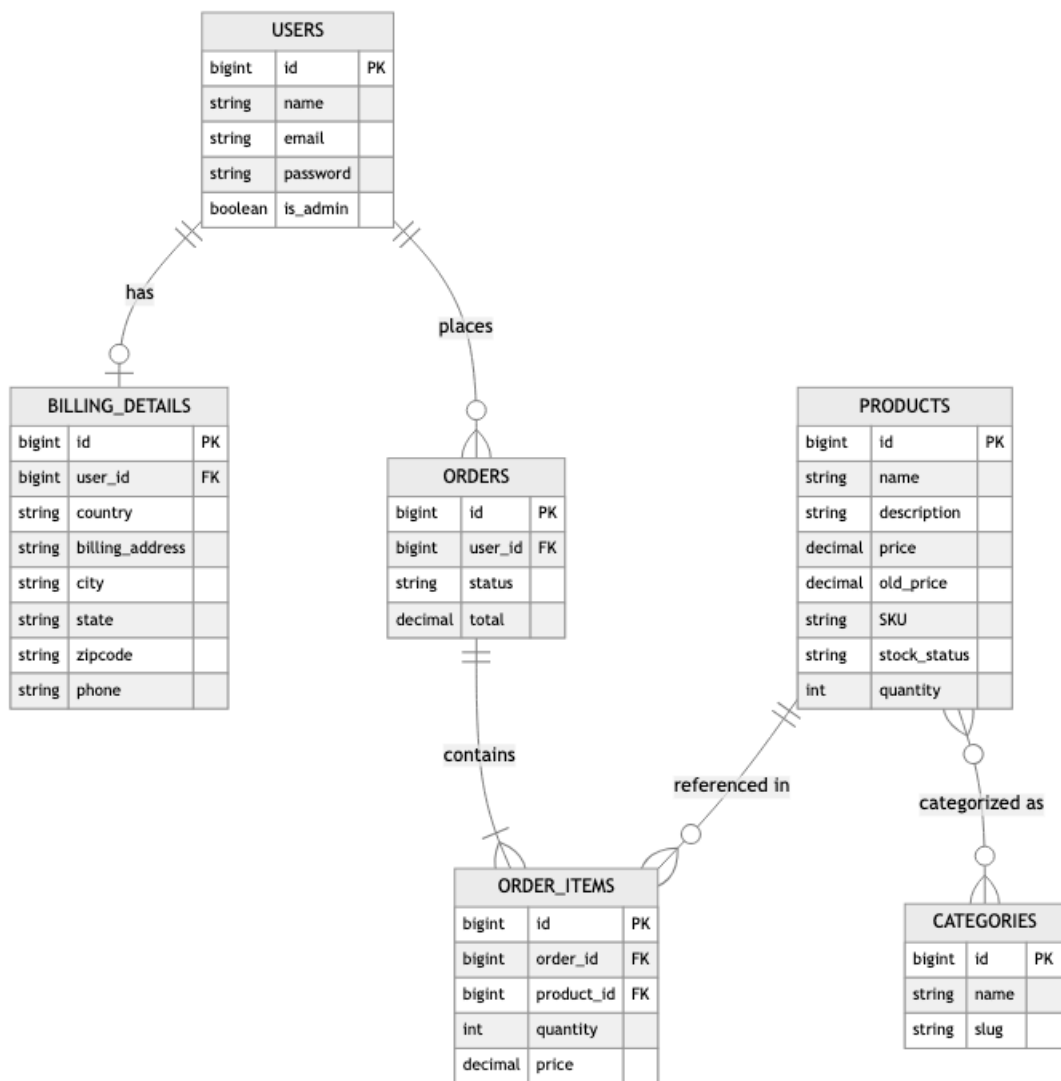


Sequence Diagram. Diagram berikut menggambarkan alur proses login yang dilanjutkan dengan pembuatan pesanan (checkout), termasuk dua kemungkinan hasil: stok mencukupi atau tidak mencukupi.



2.4 Skema Diagram Relasi Tabel

Skema basis data terdiri atas enam tabel inti yang dikelola melalui Eloquent ORM dan migration Laravel (folder `database/migrations/`). Diagram Entity-Relationship berikut menggambarkan seluruh tabel beserta kolom dan relasinya.



Tabel	Kolom Utama	Relasi
users	name, email, password, is_admin	hasMany Order, hasOne BillingDetail
categories	name, slug	belongsToMany Product (melalui tabel pivot category_product)
products	name, description, price, old_price, SKU, stock_status, quantity, image, images	belongsToMany Category
orders	user_id, status, total, session_id	belongsTo User, hasMany OrderItem
order_items	order_id, product_id, quantity, price	belongsTo Order, belongsTo Product
billing_details	user_id, country, billing_address, city, state, zipcode, phone, order_notes	belongsTo User

2.5 Model

Setiap tabel direpresentasikan oleh satu model Eloquent yang mendefinisikan atribut yang boleh diisi (mass-assignment) serta relasinya ke model lain. Berikut contoh migration dan model untuk tabel `orders`.

```
Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('user_id');
    $table->enum('status', ['pending', 'processing', 'completed', 'canceled']);
    $table->decimal('total', 8, 2);
    $table->string('session_id')->nullable();
    $table->timestamps();
    $table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');
});
```

```
class Order extends Model
{
    use HasFactory;
    protected $fillable = ['user_id', 'status', 'total', 'session_id'];

    public function user() { return $this->belongsTo(User::class); }
    public function orderItems() { return $this->hasMany(OrderItem::class); }
}
```

Catatan: setiap model menggunakan properti `$fillable` untuk mass-assignment protection, yaitu mencegah kolom yang tidak diinginkan terisi lewat request pengguna. Foreign key pada migration juga memakai `onDelete('cascade')` agar data anak (misalnya `order_items` dan baris pada `category_product`) ikut terhapus ketika data induknya dihapus, sehingga integritas referensial basis data tetap terjaga.

2.6 Query

Seluruh query database dibangun melalui Eloquent Query Builder, bukan menuliskan SQL mentah secara langsung. Pendekatan ini secara otomatis melakukan *parameter binding* pada setiap nilai yang disisipkan ke query, sehingga input pengguna tidak pernah digabungkan langsung ke teks SQL — inilah mekanisme utama yang mencegah celah SQL Injection. Untuk membuktikan hal ini secara konkret, query yang benar-benar dieksekusi oleh MySQL direkam menggunakan `DB::enableQueryLog()` saat memanggil endpoint `GET /products?category=mawar&min_price=50000&sort=price_asc`:

SQL asli yang dieksekusi MySQL (hasil query log)

```
select count(*) as aggregate from `products`
where exists (select * from `categories`
              inner join `category_product` on `categories`.`id` =
`category_product`.`category_id`
              where `products`.`id` = `category_product`.`product_id` and `categories`.`slug` =
?)
and `price` >= ?
-- bindings: ["mawar", 50000]

select * from `products`
where exists (select * from `categories`
              inner join `category_product` on `categories`.`id` =
`category_product`.`category_id`
              where `products`.`id` = `category_product`.`product_id` and `categories`.`slug` =
?)
and `price` >= ?
order by `price` asc limit 5 offset 0
-- bindings: ["mawar", 50000]

select          `categories`.*,          `category_product`.`product_id`,
`category_product`.`category_id`
from `categories` inner join `category_product` on `categories`.`id` =
`category_product`.`category_id`
where `category_product`.`product_id` in (1, 8)
-- bindings: []
```

Tanda tanya (?) pada query di atas adalah placeholder yang nilainya dikirim terpisah lewat parameter `bindings`, bukan disisipkan langsung ke teks SQL — sehingga meskipun pengguna memasukkan karakter berbahaya seperti tanda kutip atau titik koma, nilai tersebut akan selalu diperlakukan sebagai data biasa, bukan sebagai perintah SQL tambahan. Query ketiga adalah query tambahan untuk memuat relasi `categories` (dipicu oleh pemanggilan `with('categories')` pada controller). Teknik *eager loading* ini mencegah masalah N+1 query, yaitu kondisi di mana kategori setiap produk harus di-query satu per satu jika tidak dimuat lebih awal.

Contoh kedua, untuk endpoint `GET /orders?status=pending` yang menampilkan daftar pesanan milik pengguna yang sedang masuk:

SQL asli yang dieksekusi MySQL (hasil query log)

```
select count(*) as aggregate from `orders` where `user_id` = ? and `status` = ?
-- bindings: [1, "pending"]
```

2.7 Import CSV

Untuk kebutuhan administrator yang ingin menambahkan banyak produk sekaligus — misalnya saat migrasi data dari toko lama atau menerima daftar produk dari pemasok — disediakan endpoint `POST /api/v1/products/import` yang hanya dapat diakses oleh administrator. Endpoint ini menerima satu berkas

CSV dan melakukan *upsert* (membuat data baru atau memperbarui data yang sudah ada) berdasarkan kolom SKU sebagai kunci pencocokan.

Format berkas CSV. Kolom wajib pada baris header adalah `name, sku, price, quantity`. Kolom opsional meliputi `brief_description, description, old_price, stock_status, image, categories` (slug kategori dipisahkan dengan tanda koma). Berikut contoh isi berkas (`docs/products_import_sample.csv`):

```
name,brief_description,description,price,old_price,sku,stock_status,quantity,image,categories
Buket Anyelir Pink,Anyelir pink lembut,Rangkaian anyelir segar,95000,120000,ANY-001,instock,15,img.jpg,mawar
Buket Krisan Putih,Krisan putih elegan,Cocok acara formal,110000,140000,KRS-001,instock,10,img2.jpg,"lily,tulip"
```

Setiap baris diproses satu per satu. Apabila satu baris tidak lolos validasi, baris tersebut akan dicatat sebagai kegagalan dan dilewati, sementara baris lain pada berkas yang sama tetap diproses — sehingga satu baris yang salah tidak menggagalkan keseluruhan proses impor. Berikut cuplikan kode intinya:

```
while (($row = fgetcsv($handle)) !== false) {
    $data = array_combine($header, array_pad($row, count($header), null));
    $validator = Validator::make($data, [
        'name' => ['required', 'string', 'max:255'],
        'sku' => ['required', 'string', 'max:255'],
        'price' => ['required', 'numeric', 'min:0'],
        'quantity' => ['required', 'integer', 'min:0'],
    ]);

    if ($validator->fails()) {
        $errors[] = ['row' => $rowNumber, 'errors' => $validator->errors()->all()];
        continue; // baris invalid dilewati, proses tetap berlanjut ke baris berikutnya
    }

    $existed = Product::where('SKU', $valid['sku']->exists());
    $product = Product::updateOrCreate(['SKU' => $valid['sku']], [ /* kolom lain */ ]);
    $existed ? $updated++ : $imported++;

    if (! empty($valid['categories'])) {
        $categoryIds = Category::whereIn('slug', explode(',', $valid['categories']))->pluck('id');
        $product->categories()->syncWithoutDetaching($categoryIds);
    }
}
```

Endpoint ini diuji langsung terhadap server yang benar-benar berjalan, menggunakan berkas sampel di atas dan token milik akun administrator:

curl -F "file=@docs/products_import_sample.csv" (menggunakan token administrator)

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message": "Import completed.", "summary": {"created": 2, "updated": 0, "failed": 0, "errors": []}}
```

Dua produk baru (ANY-001 dan KRS-001) berhasil dibuat pada basis data MySQL sungguhan dalam percobaan ini, kemudian dibersihkan kembali setelah pengujian selesai. Endpoint ini juga dilindungi middleware `admin.api`; pengguna biasa yang mencoba mengakses akan menerima status `403`, sedangkan permintaan tanpa token menerima `401` (lihat pembahasan pada Bagian 2.12 Unit Testing).

2.8 API

Seluruh endpoint API berada di bawah prefix `/api/v1`, didaftarkan pada berkas `routes/api.php`, dan diproses oleh kumpulan controller khusus pada namespace `App\Http\Controllers\Api` — dipisahkan dari controller storefront Livewire agar API bersifat stateless sepenuhnya.

Method	Endpoint	Auth	Keterangan
POST	<code>/api/v1/auth/register</code>	Tidak	Registrasi pengguna baru, langsung menerima token
POST	<code>/api/v1/auth/login</code>	Tidak	Masuk (login), menerima bearer token
POST	<code>/api/v1/auth/logout</code>	Ya	Mencabut token yang sedang dipakai
GET	<code>/api/v1/auth/me</code>	Ya	Menampilkan profil pengguna yang sedang masuk
GET	<code>/api/v1/categories</code>	Tidak	Daftar kategori (pagination)
GET	<code>/api/v1/categories/{id}</code>	Tidak	Detail satu kategori
GET	<code>/api/v1/products</code>	Tidak	Daftar produk (pagination, filter, pencarian, sorting)
GET	<code>/api/v1/products/{id}</code>	Tidak	Detail satu produk
POST	<code>/api/v1/products/import</code>	Ya (admin)	Impor massal produk dari berkas CSV
GET	<code>/api/v1/orders</code>	Ya	Daftar pesanan milik pengguna (admin melihat semua)
POST	<code>/api/v1/orders</code>	Ya	Membuat pesanan (checkout) dengan validasi stok
GET	<code>/api/v1/orders/{id}</code>	Ya	Detail satu pesanan (hanya pemilik atau admin)

Berikut definisi routing lengkap sebagaimana didaftarkan pada `routes/api.php`:

```

Route::prefix('v1')->group(function () {
    Route::post('/auth/register', [AuthController::class, 'register'])->
    >middleware('throttle:6,1');
    Route::post('/auth/login', [AuthController::class, 'login'])->
    >middleware('throttle:6,1');

    Route::middleware('auth:sanctum')->group(function () {
        Route::post('/auth/logout', [AuthController::class, 'logout']);
        Route::get('/auth/me', [AuthController::class, 'me']);
    });

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/categories/{category}', [CategoryController::class, 'show']);
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{product}', [ProductController::class, 'show']);

    Route::middleware(['auth:sanctum', 'admin.api'])->group(function () {
        Route::post('/products/import', [ProductImportController::class, 'import']);
    });

    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/orders', [OrderController::class, 'index']);
        Route::post('/orders', [OrderController::class, 'store']);
        Route::get('/orders/{order}', [OrderController::class, 'show']);
    });
});

```

Sebagai contoh alur transaksi utama, berikut bukti nyata permintaan pembuatan pesanan (checkout) beserta responsnya:

HTTP/1.1 201 Created

```

POST /api/v1/orders
Authorization: Bearer <token>
Content-Type: application/json

{ "items": [ { "product_id": 7, "quantity": 2 } ],
  "billing": { "country": "Indonesia", "billing_address": "Jl. Contoh No.1",
               "city": "Bandung", "state": "Jawa Barat", "zipcode": "40123",
               "phone": "081234567890" } }

Response:
{"message":"Order created successfully.", "data":
{"id":5, "status":"pending", "total":50000,
  "items":[{"id":5, "product_id":7, "product_name":"ZZZ Demo Product
Laporan", "quantity":2,
  "price":25000, "subtotal":50000}], ...}}

```

2.9 Auth

Autentikasi API menggunakan Laravel Sanctum dengan skema *Personal Access Token*, berbeda dengan storefront Livewire yang menggunakan cookie/sesi. Dengan skema token, API menjadi sepenuhnya stateless — server tidak perlu menyimpan status sesi apa pun antar permintaan, sehingga cocok dikonsumsi oleh aplikasi mobile maupun aplikasi web (SPA) di luar domain storefront.

- Endpoint `POST /auth/register` dan `POST /auth/login` masing-masing mengembalikan token dalam bentuk string Bearer setelah proses registrasi/masuk berhasil.
- Setiap endpoint privat dilindungi middleware `auth:sanctum` yang mewajibkan header `Authorization: Bearer <token>` pada setiap permintaan.
- Endpoint khusus administrator (`/products/import`) mendapat lapisan tambahan berupa middleware `admin.api` (didefinisikan pada `app/Http/Middleware/EnsureApiUserIsAdmin.php`).
- Permintaan tanpa token akan ditolak dengan status `401 Unauthenticated`. Token yang valid namun digunakan untuk mengakses data milik pengguna lain akan ditolak dengan status `403 Forbidden` — diperiksa lewat `app/Policies/OrderPolicy.php`.

```
class OrderPolicy
{
    public function view(User $user, Order $order): bool
    {
        return $user->is_admin || $user->id === $order->user_id;
    }
}
```

GET /auth/me — perbandingan tanpa dan dengan token

```
Tanpa token -> HTTP/1.1 401 Unauthorized {"message":"Unauthenticated."}
Dengan token -> HTTP/1.1 200 OK {"user":{"id":6,"name":"Demo
User","email":"...", "is_admin":false}}
```

2.10 Throttling, Pagination, Filtering, Sorting

Throttling (Pembatasan Jumlah Permintaan)

Throttling adalah mekanisme yang membatasi berapa kali satu klien boleh memanggil suatu endpoint dalam rentang waktu tertentu. Cara kerjanya: setiap permintaan yang masuk akan melewati middleware `ThrottleRequests`, yang mengidentifikasi klien berdasarkan ID pengguna (jika sudah masuk) atau alamat IP (jika masih tamu), lalu mencatat jumlah permintaan klien tersebut dalam jendela waktu satu menit menggunakan cache. Jika jumlah permintaan pada jendela waktu tersebut belum melampaui batas, permintaan diteruskan ke controller dan penghitung ditambah satu. Jika sudah melampaui batas, permintaan langsung ditolak dengan status `429 Too Many Requests` tanpa sempat menjalankan logic controller sama sekali — sehingga controller (dan basis data) tidak pernah terbebani oleh permintaan yang melebihi kuota.

Pada backend TokoJoko, throttling diterapkan dalam dua lapis:

Cakupan	Batas	Lokasi Definisi
Seluruh <code>/api/*</code>	60 permintaan/menit per pengguna (atau per IP untuk tamu)	<code>app/Providers/RouteServiceProvider.php</code> , diterapkan otomatis lewat grup middleware <code>api</code> pada <code>app/Http/Kernel.php</code>
<code>/auth/register</code> dan <code>/auth/login</code>	6 permintaan/menit per IP (tambahan, lebih ketat)	didefinisikan langsung pada route di <code>routes/api.php</code>

```
// app/Providers/RouteServiceProvider.php - limiter global 60/menit
RateLimiter::for('api', function (Request $request) {
    return Limit::perMinute(60)->by($request->user()?->id ?: $request->ip());
});

// routes/api.php - limiter tambahan 6/menit khusus endpoint auth
Route::post('/auth/register', [...])->middleware('throttle:6,1');
Route::post('/auth/login', [...])->middleware('throttle:6,1');
```

Limiter tambahan pada `/auth/login` dan `/auth/register` sengaja dibuat lebih ketat karena kedua endpoint tersebut adalah target paling umum bagi serangan *brute-force* (mencoba banyak kombinasi kata sandi) maupun pembuatan akun secara masal oleh bot. Untuk membuktikan mekanisme ini benar-benar berjalan, dilakukan delapan kali percobaan `POST /auth/login` secara berurutan dengan kata sandi yang salah, langsung ke server yang sedang berjalan:

&x curl -i POST /auth/login (kata sandi salah, dikirim berurutan)

```
Percobaan ke-1 s.d. ke-6 -> HTTP/1.1 422 | X-RateLimit-Limit: 6 | "The provided
credentials are incorrect."
Percobaan ke-7 dan ke-8 -> HTTP/1.1 429 | X-RateLimit-Limit: 60 | "Too Many
Attempts."
```

Enam percobaan pertama masih diproses controller (ditolak karena kredensial salah, status 422), tetapi tepat pada percobaan ketujuh — setelah kuota 6 permintaan/menit habis — permintaan langsung diblokir dengan status 429 sebelum sempat mencapai controller. Pengujian ini membuktikan *throttling* bekerja sesuai rancangan, bukan sekadar klaim di dokumen.

Pagination

Pagination adalah teknik membagi hasil query yang jumlahnya banyak menjadi beberapa halaman kecil, sehingga server tidak perlu memuat dan mengirim seluruh data sekaligus dalam satu respons. Backend ini menggunakan method `paginate()` bawaan Eloquent, yang secara otomatis melakukan tiga hal: (1) menghitung jumlah total data yang cocok dengan query menggunakan `SELECT COUNT(*)`, (2) mengambil hanya sebagian data sejumlah `per_page` menggunakan klausa SQL `LIMIT` dan `OFFSET`, dan (3) mengembalikan metadata halaman (`current_page`, `last_page`, `per_page`, `total`) agar klien — misalnya aplikasi mobile — dapat menampilkan navigasi antar halaman.

```
{ "data": [ ... ], "meta": { "current_page": 1, "last_page": 2, "per_page": 3, "total":
5 } }
```

Nilai default `per_page` pada seluruh endpoint daftar (produk, kategori, dan pesanan) diatur ke **3** data per halaman, dan dapat diperbesar oleh klien lewat parameter `?per_page=` hingga maksimum 50 (dibatasi lewat `max(1, min($perPage, 50))` pada controller agar klien tidak dapat meminta seluruh data sekaligus dan membebani server). Berikut bukti nyata perbandingan halaman pertama dan kedua pada endpoint `GET /products` tanpa parameter tambahan apa pun (basis data pengujian berisi 5 produk):

GET /products (halaman 1, tanpa parameter — memakai nilai default per_page=3)

```
meta: { "current_page": 1, "last_page": 2, "per_page": 3, "total": 5 }
data (id produk yang muncul): [1, 2, 3]
```

GET /products?page=2 (halaman kedua)

```
meta: { "current_page": 2, "last_page": 2, "per_page": 3, "total": 5 }
data (id produk yang muncul): [4, 5]
```

Dari 5 produk yang tersedia, halaman pertama menampilkan 3 produk (id 1-3) dan halaman kedua menampilkan 2 produk sisanya (id 4-5) — sesuai dengan nilai `per_page=3` dan `last_page=2` yang dilaporkan pada metadata.

Filtering dan Sorting

Khusus endpoint `GET /products`, tersedia empat parameter query tambahan yang dapat dikombinasikan untuk menyaring dan mengurutkan hasil sebelum data dipotong menjadi halaman-halaman oleh mekanisme pagination di atas.

Parameter	Contoh	Efek
<code>category</code>	<code>?category=mawar</code>	Menyaring produk berdasarkan slug kategori, memakai <code>whereHas</code> pada relasi many-to-many.
<code>search</code>	<code>?search=rose</code>	Mencari kata kunci pada kolom <code>name</code> dan <code>description</code> menggunakan <code>LIKE</code> .
<code>min_price</code> / <code>max_price</code>	<code>?min_price=10&max_price=100</code>	Menyaring produk pada rentang harga tertentu.
<code>sort</code>	<code>?sort=price_asc</code>	Mengurutkan hasil: <code>latest</code> (terbaru), <code>price_asc</code> (harga naik), atau <code>price_desc</code> (harga turun).

Cuplikan kode pada `app/Http/Controllers/Api/ProductController.php` yang menyusun keempat parameter tersebut menjadi satu query Eloquent:

```

if ($category = $request->query('category')) {
    $query->whereHas('categories', fn ($q) => $q->where('categories.slug', $category));
}
if ($search = $request->query('search')) {
    $query->where(fn ($q) => $q->where('name', 'like', "%{$search}%")-
    >orWhere('description', 'like', "%{$search}%"));
}
match ($request->query('sort')) {
    'price_asc' => $query->orderBy('price', 'asc'),
    'price_desc' => $query->orderBy('price', 'desc'),
    'latest' => $query->orderBy('created_at', 'desc'),
    default => $query->orderBy('id', 'asc'),
};

```

2.11 Docs

Dokumentasi API disediakan dalam tiga bentuk yang saling melengkapi, agar dapat digunakan baik untuk pembacaan cepat maupun pengujian interaktif.

1. **Markdown reference** — berkas `docs/API.md`, berisi deskripsi setiap endpoint beserta contoh permintaan dan respons.
2. **Swagger UI interaktif** — dapat diakses pada rute `/docs`, menampilkan spesifikasi OpenAPI 3.0 dari `public/openapi.json` (kini turut mencakup endpoint `/products/import`), lengkap dengan tombol Authorize untuk mencoba endpoint privat langsung dari peramban.
3. **Postman Collection** — berkas `docs/postman_collection.json`, yang dapat langsung diimpor ke Postman dan secara otomatis menyimpan token ke variabel koleksi setelah permintaan Login/Register berhasil, termasuk permintaan untuk uji impor CSV.

2.12 Unit Testing

Pengujian otomatis (unit dan feature test) memakai PHPUnit bawaan Laravel, dan dijalankan di atas basis data SQLite in-memory (dikonfigurasi pada `phpunit.xml`) — sehingga proses pengujian tidak pernah menyentuh basis data MySQL yang dipakai untuk pengembangan.

Berkas Pengujian	Cakupan
<code>AuthApiTest.php</code>	Registrasi, login, email duplikat, kata sandi salah, rate limit, profil (me), logout (8 pengujian)
<code>ProductApiTest.php</code>	Pagination, filter kategori, pencarian, filter harga, sorting, detail, 404 (7 pengujian)
<code>CategoryApiTest.php</code>	Daftar, detail, 404 (3 pengujian)
<code>OrderApiTest.php</code>	Tamu ditolak, checkout berhasil dan stok berkurang, stok tidak cukup menghasilkan 422, isolasi data per pengguna, 403 untuk pesanan orang lain, admin dapat melihat semua (6 pengujian)
<code>ProductImportApiTest.php</code>	Tamu 401, non-admin 403, admin berhasil impor sekaligus melekatkan kategori, upsert berdasarkan SKU, baris tidak valid dilewati, header CSV kurang lengkap menghasilkan 422 (6 pengujian)

```
php artisan test          # menjalankan seluruh test suite (54 pengujian)
php artisan test --filter=Api  # menjalankan 30 pengujian khusus API
```

Hasil eksekusi "php artisan test" (dijalankan ulang saat laporan ini disusun)

```
PASS Tests\Unit\ExampleTest (1 pengujian)
PASS Tests\Feature\Api\AuthApiTest (8 pengujian)
PASS Tests\Feature\Api\CategoryApiTest (3 pengujian)
PASS Tests\Feature\Api\OrderApiTest (6 pengujian)
PASS Tests\Feature\Api\ProductApiTest (7 pengujian)
PASS Tests\Feature\Api\ProductImportApiTest (6 pengujian)
PASS Tests\Feature\Auth\* (bawaan Breeze, 15 pengujian)
PASS Tests\Feature\ExampleTest (1 pengujian)
PASS Tests\Feature\ProfileTest (5 pengujian)

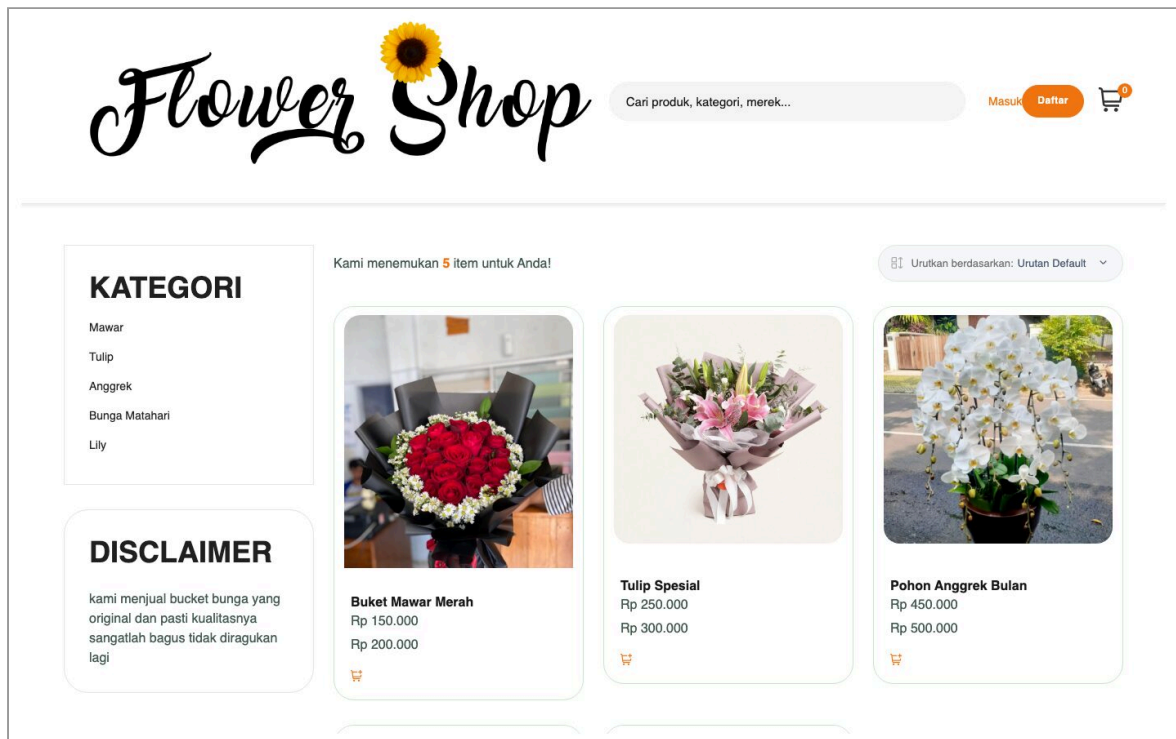
Tests:    54 passed (141 assertions)
Duration: 3.70s
```

BAB III

IMPLEMENTASI SISTEM

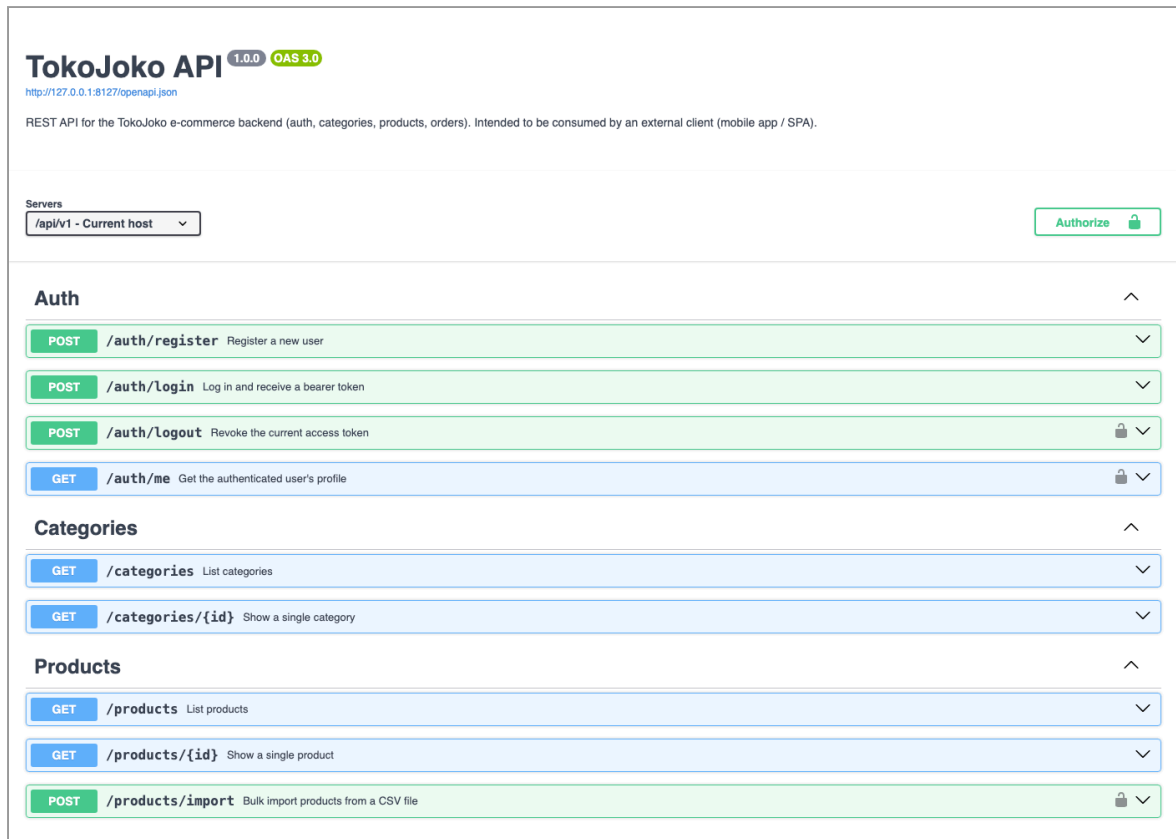
Tangkapan layar berikut diambil langsung dari server yang benar-benar dijalankan (`php artisan serve`) di atas basis data MySQL pengembangan, sehingga membuktikan sistem berjalan secara nyata dan bukan sekadar rancangan di atas kertas.

3.1 Storefront (Livewire) — Halaman Utama



Halaman utama menampilkan katalog produk (nama, harga, kategori) yang datanya sama persis dengan yang disajikan lewat REST API `GET /api/v1/products`.

3.2 Swagger UI — Dokumentasi API Interaktif



Seluruh 12 endpoint tergabung berdasarkan tag (*Auth*, *Categories*, *Products*, *Orders*), termasuk endpoint `/products/import` yang baru ditambahkan, lengkap dengan ikon gembok pada endpoint yang membutuhkan autentikasi.

3.3 Menjalankan Server (untuk Keperluan Presentasi/Demo)

```
composer install
cp .env.example .env
php artisan key:generate
# atur DB_DATABASE / DB_USERNAME / DB_PASSWORD pada berkas .env
php artisan migrate
php artisan storage:link
php artisan serve
```

Server akan berjalan pada `http://localhost:8000`, dengan dokumentasi interaktif dapat diakses pada `http://localhost:8000/docs`.

4.1 Kesimpulan

Berdasarkan hasil analisa, perancangan, implementasi, dan pengujian yang telah diuraikan pada bab-bab sebelumnya, dapat disimpulkan bahwa seluruh ruang lingkup tugas telah terpenuhi sebagai berikut.

1	Desain basis data, model, dan schema — enam tabel beserta migration dan model Eloquent lengkap dengan relasinya (Bagian 2.4-2.5).
2	Query — dibuktikan melalui SQL asli hasil query log, aman dari SQL Injection berkat parameter binding, dan menghindari N+1 query melalui eager loading (Bagian 2.6).
3	Import CSV — endpoint khusus admin, upsert berdasarkan SKU, baris tidak valid dilewati tanpa menggagalkan seluruh berkas, telah diuji terhadap server sungguhan (Bagian 2.7).
4	API — 12 endpoint REST bersifat stateless untuk kebutuhan autentikasi, produk, kategori, pesanan, dan impor data (Bagian 2.8).
5	Auth — autentikasi token Sanctum disertai otorisasi kepemilikan data (Bagian 2.9).
6	Throttling, pagination, filtering, sorting — pembatasan permintaan dua lapis yang telah dibuktikan memblokir permintaan berlebih (429), pagination dengan default 3 data per halaman yang telah dibuktikan membagi data dengan benar, serta filter dan pengurutan produk (Bagian 2.10).
7	Docs — dokumentasi Markdown, Swagger UI, dan Postman Collection (Bagian 2.11).
8	Unit testing — 54 pengujian otomatis (30 di antaranya khusus API), seluruhnya berstatus lulus (Bagian 2.12).

Seluruh kebutuhan fungsional dan non-fungsional yang dirumuskan pada Bagian 2.1 telah dipenuhi dan diverifikasi, baik melalui pengujian otomatis (PHPUnit) maupun pengujian manual langsung terhadap server yang sedang berjalan.

4.2 Saran Pengembangan Lanjutan

- Menambahkan mekanisme refresh token atau kedaluwarsa token otomatis agar sesi API lebih aman untuk pemakaian jangka panjang pada aplikasi mobile.
- Menyediakan endpoint unggah berkas gambar produk secara langsung (multipart), bukan hanya menerima path berupa string, agar administrator tidak perlu mengelola berkas secara manual.
- Menambahkan halaman admin pada panel Filament untuk memicu proses impor CSV melalui antarmuka pengguna, bukan hanya melalui API/Postman.
- Menerapkan queue (antrean tugas) untuk proses impor CSV berukuran besar agar tidak memblokir permintaan HTTP saat diproses.
- Mempersempit konfigurasi `config/cors.php` agar hanya mengizinkan origin klien resmi sebelum sistem dirilis ke lingkungan produksi.

LAMPIRAN CODE PROGRAM

Kode sumber lengkap tersedia pada repository (https://github.com/imdevedugame/e-commerce_new).
Berikut cuplikan berkas-berkas inti dari implementasi backend REST API.

L.1 routes/api.php

```
<?php

use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\CategoryController;
use App\Http\Controllers\Api\OrderController;
use App\Http\Controllers\Api\ProductController;
use App\Http\Controllers\Api\ProductImportController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    return $request->user();
});

Route::prefix('v1')->group(function () {
    Route::post('/auth/register', [AuthController::class, 'register'])->
    >middleware('throttle:6,1');
    Route::post('/auth/login', [AuthController::class, 'login'])->
    >middleware('throttle:6,1');

    Route::middleware('auth:sanctum')->group(function () {
        Route::post('/auth/logout', [AuthController::class, 'logout']);
        Route::get('/auth/me', [AuthController::class, 'me']);
    });

    Route::get('/categories', [CategoryController::class, 'index']);
    Route::get('/categories/{category}', [CategoryController::class, 'show']);
    Route::get('/products', [ProductController::class, 'index']);
    Route::get('/products/{product}', [ProductController::class, 'show']);

    Route::middleware(['auth:sanctum', 'admin.api'])->group(function () {
        Route::post('/products/import', [ProductImportController::class, 'import']);
    });

    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/orders', [OrderController::class, 'index']);
        Route::post('/orders', [OrderController::class, 'store']);
        Route::get('/orders/{order}', [OrderController::class, 'show']);
    });
});
```

L.2 app/Http/Controllers/Api/ProductController.php

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Resources\ProductResource;
use App\Models\Product;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index(Request $request): JsonResponse
    {
        $query = Product::query()->with('categories');

        if ($category = $request->query('category')) {
            $query->whereHas('categories', function ($q) use ($category) {
                $q->where('categories.slug', $category);
            });
        }

        if ($search = $request->query('search')) {
            $query->where(function ($q) use ($search) {
                $q->where('name', 'like', "%{$search}%")
                    ->orWhere('description', 'like', "%{$search}%");
            });
        }

        if ($request->filled('min_price')) {
            $query->where('price', '>=', (float) $request->query('min_price'));
        }
        if ($request->filled('max_price')) {
            $query->where('price', '<=', (float) $request->query('max_price'));
        }

        match ($request->query('sort')) {
            'price_asc' => $query->orderBy('price', 'asc'),
            'price_desc' => $query->orderBy('price', 'desc'),
            'latest' => $query->orderBy('created_at', 'desc'),
            default => $query->orderBy('id', 'asc'),
        };

        $perPage = max(1, min((int) $request->query('per_page', 3), 50));
        $products = $query->paginate($perPage)->appends($request->query());

        return response()->json([
            'data' => ProductResource::collection($products->items()),
            'meta' => [
                'current_page' => $products->currentPage(),
                'last_page' => $products->lastPage(),
                'per_page' => $products->perPage(),
                'total' => $products->total(),
            ],
        ]);
    }

    public function show(Product $product): JsonResponse
```

```

    {
        $product->load('categories');
        return response()->json(['data' => new ProductResource($product)]);
    }
}

```

L.3 app/Http/Controllers/Api/OrderController.php (method store — checkout)

```

public function store(StoreOrderRequest $request): JsonResponse
{
    $order = DB::transaction(function () use ($request) {
        $user = $request->user();
        $items = $request->validated('items');

        $products = Product::query()->whereIn('id', array_column($items, 'product_id'))
            ->lockForUpdate()->get()->keyBy('id');

        $total = 0;
        $orderItemsData = [];
        foreach ($items as $item) {
            $product = $products->get($item['product_id']);
            if ($product->quantity < $item['quantity']) {
                throw ValidationException::withMessages([
                    'items' => ["Insufficient stock for product '{$product->name}'.
Available: {$product->quantity}."],
                ]);
            }
            $subtotal = $product->price * $item['quantity'];
            $total += $subtotal;
            $orderItemsData[] = ['product' => $product, 'quantity' => $item['quantity'],
'price' => $product->price];
        }

        $order = Order::create(['user_id' => $user->id, 'status' => 'pending', 'total'
=> $total]);

        foreach ($orderItemsData as $data) {
            OrderItem::create([
                'order_id' => $order->id, 'product_id' => $data['product']->id,
                'quantity' => $data['quantity'], 'price' => $data['price'],
            ]);
            $data['product']->decrement('quantity', $data['quantity']);
            if ($data['product']->fresh()->quantity <= 0) {
                $data['product']->update(['stock_status' => 'outstock']);
            }
        }

        BillingDetail::updateOrCreate(['user_id' => $user->id, $request-
>validated('billing')]);
        return $order;
    });

    $order->load('orderItems.product');
    return response()->json(['message' => 'Order created successfully.', 'data' => new
OrderResource($order)], 201);
}

```

L.4 app/Http/Controllers/Api/ProductImportController.php

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Category;
use App\Models\Product;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Validator;

class ProductImportController extends Controller
{
    private const REQUIRED_COLUMNS = ['name', 'sku', 'price', 'quantity'];

    public function import(Request $request): JsonResponse
    {
        $request->validate(['file' => ['required', 'file', 'mimes:csv,txt']]);

        $handle = fopen($request->file('file')->getRealPath(), 'r');
        $header = array_map(fn ($col) => strtolower(trim($col)), fgetcsv($handle));

        $missing = array_diff(self::REQUIRED_COLUMNS, $header);
        if (! empty($missing)) {
            fclose($handle);
            return response()->json(['message' => 'CSV header is missing required
column(s): '.implode(' ', $missing)], 422);
        }

        $imported = 0; $updated = 0; $errors = []; $rowNumber = 1;

        while (($row = fgetcsv($handle)) !== false) {
            $rowNumber++;
            if (count(array_filter($row, fn ($v) => $v !== null && $v !== '')) === 0)
                continue;

            $data = array_combine($header, array_pad($row, count($header), null));
            $validator = Validator::make($data, [
                'name' => ['required', 'string', 'max:255'],
                'sku' => ['required', 'string', 'max:255'],
                'price' => ['required', 'numeric', 'min:0'],
                'quantity' => ['required', 'integer', 'min:0'],
                'categories' => ['nullable', 'string'],
            ]);

            if ($validator->fails()) {
                $errors[] = ['row' => $rowNumber, 'errors' => $validator->errors()-
>all()];
                continue;
            }

            $valid = $validator->validated();
            DB::transaction(function () use ($valid, &$imported, &$updated) {
                $existed = Product::where('SKU', $valid['sku'])->exists();
                $product = Product::updateOrCreate(['SKU' => $valid['sku']], [
                    'name' => $valid['name'], 'price' => $valid['price'], 'quantity' =>
                    $valid['quantity'],
```

```

        'stock_status' => $valid['stock_status'] ?? 'instock', 'images' =>
json_encode([]),
    ));
    $existed ? $updated++ : $imported++;

    if (! empty($valid['categories'])) {
        $slugs = array_filter(array_map('trim', explode(',',
$valid['categories'])));
        $product->categories()-
>syncWithoutDetaching(Category::whereIn('slug', $slugs)->pluck('id'));
    }
    });
}
fclose($handle);

return response()->json([
    'message' => 'Import completed.',
    'summary' => ['created' => $imported, 'updated' => $updated, 'failed' =>
count($errors)],
    'errors' => $errors,
]);
}
}
}

```

L.5 app/Http/Middleware/EnsureApiUserIsAdmin.php

```

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureApiUserIsAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (! $request->user() || ! $request->user()->is_admin) {
            abort(403, 'This action requires an administrator account.');
```

L.6 app/Http/Controllers/Api/AuthController.php

```
<?php

namespace App\Http\Controllers\Api;

class AuthController extends Controller
{
    public function register(RegisterRequest $request): JsonResponse
    {
        $user = User::create([
            'name' => $request->validated('name'),
            'email' => $request->validated('email'),
            'password' => Hash::make($request->validated('password')),
        ]);
        $token = $user->createToken('api-token')->plainTextToken;

        return response()->json([
            'message' => 'Registration successful.',
            'user' => new UserResource($user),
            'token' => $token,
        ], 201);
    }

    public function login(LoginRequest $request): JsonResponse
    {
        if (! Auth::once($request->validated())) {
            throw ValidationException::withMessages(['email' => ['The provided
credentials are incorrect.']] );
        }
        $token = Auth::user()->createToken('api-token')->plainTextToken;
        return response()->json([
            'message' => 'Login successful.', 'user' => new UserResource(Auth::user()),
            'token' => $token,
        ]);
    }

    public function logout(Request $request): JsonResponse
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Logged out successfully.']);
    }

    public function me(Request $request): JsonResponse
    {
        return response()->json(['user' => new UserResource($request->user())]);
    }
}
```